

Transcripcion: Sesion de Preparacion para Defensa Backend

Reunion del equipo de desarrollo — Arquitectura MVC, API RESTful y Seguridad

Parte 1: Introduccion a la Arquitectura MVC

[00:00:00]

Ponente: Si, okay. Buenas noches, muchachos. Vamos a ver, como les pregunte ahorita, este... entender a grandes rasgos como va, como funciona este... lo que estamos realizando en el back. Primero, este... lo que es Laravel y muchas de las aplicaciones back como tal utilizan una cierta estructura, una cierta arquitectura llamada esto MVC. Es decir: Modelo, Vista, Controlador. Que son varias que al momento de uno crear un, por asi decirlo, un proyecto en Laravel, el siempre te crea esa estructura. Y okay, tiene esa... y cada uno de esas partes se... eh... se enfoca en algo, algo en especifico.

[00:00:46]

Ponente: Si uno va a la parte del codigo, por ejemplo aqui estoy nada mas en la parte del backend, el Laravel siempre crea los modelos, crea los controladores dentro y crea lo que son las vistas. Las vistas como tal es de lo que se encargan ustedes, los del frontend. Ustedes son los que se encargan de hacer esa parte. Pero nosotros como back nos encargamos de mandarle la informacion o los datos que va a servir esa vista. Y con este diagrama vamos a ver como es todo el flujo de practicamente todo.

[00:01:25]

Ponente: Empezando... todo empieza desde los... lo que son los input, los... son los puntos de acceso de tanto de ustedes como el punto donde nosotros exponemos este... la entrada al backend. Ustedes inician... ahorita vamos a verlo en el codigo donde esta, pero basicamente ustedes en ciertas partes de sus componentes, los componentes que ustedes crearon, colocan los endpoints o llaman ese endpoint. Ese se comunica con un controlador. El controlador tiene diversos metodos que hacen distintas cosas y ese controlador se comunica con lo que son los modelos.

[00:02:04]

Ponente: Los modelos son una representacion de lo que hicimos en el... en la base de datos, de todo esta... de todo este poco de tablas, de todo este poco de relaciones. Todas esas son... aparte de creadas aqui en una... en el modelo de la base de datos, son tambien, como quien dice, modeladas o... este... puestas en codigo dentro del... de dentro de lo que es Laravel y dentro de lo que es la estructura del programa. A traves de esos modelos es que se accede a la base de datos, se van comunicando entre ellos. La respuesta la vuelve a... a recibir el controlador y a traves del... la devuelve a la vista nuevamente.

[00:02:46]

Ponente: Eso es a grandes rasgos como funciona. Ustedes hacen una solicitud a un endpoint, ese endpoint se comunica a un controlador, ese controlador trabaja sobre los... eh... sobre los datos o sobre la estructura de un modelo que tiene tanto los datos, tanto los campos de las tablas como lo

que son las relaciones de la base de datos. Y por ultimo, esa respuesta es enviada de los... desde los... nuevamente a las vistas que ustedes tienen. A grandes rasgos asi funciona todo el back e incluso viendo sus... sus codigos... por ejemplo, eh... en el proyecto completo incluso viendolo asi, hay ciertas partes donde si he visto donde este... efectivamente se comunican al... al backend.

[00:03:40]

Ponente: Por ejemplo, en la parte esta de autentificacion y login. Aqui ustedes tienen, por lo menos el ultimo... lo ultimo... esto es un boton de tipo envio, es decir, al darle clic se envian los... se envian los datos. Y el sitio donde ustedes utilizan los endpoints seria aca. Esto es como tal un endpoint que nosotros en la base de datos hemos declarado de... perdon, que en el back hemos declarado. Si nos vamos a lo que es la ruta del propio backend a la API... ya... okay, ignoremos eso por ahora.

[00:04:23]

Ponente: Se estan comunicando justamente a este... a lo que es este que esta declarado aca, que coincide con esta misma ruta. Y como le digo, a grandes rasgos asi funciona todo el... todo lo que es el proyecto en general. Aparte de lo que es el... eh... un poco mas asi como le digo en... en general, tambien estamos trabajando con lo que es la metodologia de... bueno, estamos creando lo que es una API RESTful. Es decir, que tiene como tal eh... ciertas reglas que uno sigue, tanto de... de evolucion de datos como... de como a la hora de crear, de estructurar, perdon, los endpoints y todas estas... y todas estas rutas.

[00:05:18]

Ponente: Eh... usamos los mismos... los... se utilizan distintos metodos como tal que son los verbos HTTP, que cada uno se encarga de un... de algo distinto. Por ejemplo, hay unos...estan principalmente son cuatro: el GET, el POST, el PUT y el DELETE. Cada uno se encarga de hacer algo especifico. El GET se encarga de obtener los datos, es decir, solamente de extraerlos. El POST es cuando ustedes envian informacion al servidor y ese servidor como tal lo guarda en la base de datos, ese es el POST. El PUT se encarga de modificar algun... de modificar los registros y el DELETE se encarga de eliminar.

[00:06:09]

Ponente: En general todo lo que es del backend funciona de esa forma. Como les digo, ustedes conociendo o viendo como es esta estructura uno ya sabe que es lo que hacen. Ustedes entran por aca y las vistas como tal se comunican a traves de los propios... eh... si nos vamos un poco a lo que es como tal a lo que ustedes acceden como a los metodos como tal, dependiendo de cada uno de los este... de los modulos que uno define en la base... del que uno define en el back es lo mismo.

Parte 2: Metodos HTTP, Codigos de Respuesta y Modelos

[00:07:08]

Ponente: El metodo GET se encarga de simplemente devolver informacion. Se lo devolvemos a ustedes en formato JSON y ese JSON ustedes lo reciben y lo pintan como tal en lo que es toda la estructura del front. Ese es uno de los metodos. El otro metodo es POST, que es el de creacion. Primero, a pesar de que ustedes pueden poner los datos que uno quiera, por eso mucho hincapie en seguir como esta la estructura de la base de datos. Porque a pesar de que ustedes puedan poner en los campos cualquier tipo de datos que quieren ingresar, en el backend nosotros

validamos que se esten ingresando datos especificos.

[00:07:43]

Ponente: Por ejemplo, aqui estamos en las direcciones. Cuando se quiere agregar una direccion en la base de datos -que esto lo hace el administrador- nosotros definimos que el administrador, para registrar una direccion, necesita colocar un estado, necesita colocar una ciudad, un sector, una calle y una sede. Y toda esa informacion es obligatoria. Si ustedes en un formulario de creacion de la direccion les falta uno y al momento de enviarlo al backend, eso va a dar un error porque va a decir: 'Mira, no me estas enviando toda la informacion necesaria, no te puedo registrar'.

[00:08:25]

Ponente: Y si le envian informacion de mas tambien va a dar un error (o a veces no lo da, pero puede dar un error) porque tambien esto esta ligado a lo que son los modelos, y los modelos son como tal las representaciones de los que estan en la base de datos. Por lo menos el POST se encarga siempre de dos cosas: de validar lo que entra (es decir, lo que ustedes estan enviando desde el frontend) y por ultimo retornar una respuesta. Esa respuesta tiene ciertos codigos. Por lo menos cuando no hay internet da el error 404 (que no se puede conectar o no consiguio ese endpoint).

[00:09:10]

Ponente: A veces cuando ve otro tipo de errores como los errores de 500, eso es problema de nosotros; es decir, algo paso en el backend que no esta respondiendo bien. Y los errores del 200 en adelante (del 200 hasta el 300) son codigos de confirmacion de que se enviaron bien. El codigo 200 es que todo esta perfecto, pero especificamente cada uno de estos codigos tiene una respuesta: por lo menos el codigo 201 es el codigo especifico de la creacion, codigo 422 especifico de un error de validacion, y asi funcionan todos.

[00:09:51]

Ponente: El otro POST... bueno, este es otro metodo. Como pueden ver, un mismo endpoint puede tener metodos repetidos (o sea, verbos repetidos) porque puede devolver varias cosas. Y este es el otro que le estaba diciendo: el PUT, que es el de modificacion. Igual, cuando se va a modificar es lo mismo que cuando se va a guardar: se validan todos los datos, que todos los datos esten correctamente, y por ultimo se procede a hacer la actualizacion y el guardado, y avisarles a ustedes que se guardo todo. Y el metodo de eliminacion (DELETE) que tambien sigue siendo parecido; simplemente elimina el registro en la base de datos como tal.

[00:10:37]

Ponente: Conociendo eso, ya en general sabemos como funciona todo lo que es la parte del backend. Es decir, ustedes a traves de... como les digo nuevamente, eso es lo mas importante: a traves de su componente en una cierta parte se van a comunicar a traves de un endpoint, va a llegar a este controlador, ese controlador dependiendo de lo que ustedes soliciten les va a responder de una u otra manera. Y esa respuesta se las va a enviar a ustedes nuevamente junto con un codigo que va a decir si fue exitoso o algo fallo.

[00:11:19]

Ponente: La otra parte que les estaba hablando es la parte de los modelos. Basicamente los modelos, como les digo, son representaciones de lo que son las tablas en la base de datos. Por lo menos este es el modelo de Activos. Si nos vamos a la base de datos, el modelo de activos tiene

estos campos: un articulo ID, un estado (ese estado se refiere a si esta operativo, en mantenimiento, fuera de servicio o dado de baja), una ubicacion y un valor. Y todos esos datos son representados en el modelo porque esta trabajando sobre esa tabla en esa base de datos.

[00:12:07]

Ponente: Bueno, tiene otras características esto, pero esto ya nos concierne a nosotros que son para evitar errores. A nivel de back, por lo menos hay ciertas cosas que uno no puede enviar; por lo menos un estado no podemos enviarle todos los estados, sino que 'casteamos' o transformamos una lista a puro texto. El valor puede ser un entero pero realmente necesitamos un flotante, etcetera. O sea, estas son cosas que nos conciernen a nosotros como backend para que los datos se guarden correctamente en lo que es la base de datos como tal.

[00:12:42]

Ponente: Y aquí están todas las relaciones, que como vemos aquí tiene varias. Por lo menos el activo tiene relaciones con artículos, tiene relación con ubicación, se va también para el calendario, se va para los reportes, se va para el mantenimiento... O sea, todas esas relaciones nosotros como backend tenemos que definir las, y aquí están definidas.

[00:13:10]

Ponente: Y como les digo, hasta donde llegan ustedes y donde más les interesa es hasta los modelos, porque en los modelos es donde como tal se hace todo el proceso que ustedes necesitan para devolverles al final la información que ustedes van a utilizar para pintar el front. Y así funciona en general todo el backend.

[00:13:57]

Ponente: Si el profesor les pregunta: 'Okay, por lo menos el formulario de reporte de usuario, a nivel de backend donde está eso? Explicame todo el proceso desde el frontend hasta el back'. Eso le concierne a... por ejemplo, el módulo de mantenimiento reportes acá. Ustedes buscan el que hizo la parte de reportes o el formulario del reporte, ya sabe donde estaría. Y justamente en esa parte donde dice o donde se refiere al endpoint al que se conecta, pues ya sabes que de ese endpoint se va a ir a esta ruta (porque aquí están definidas todas las rutas). Esa ruta lo va a llevar a un controlador.

[00:14:55]

Ponente: Ese controlador tiene muchos métodos. ¿Cuál método específicamente está buscando para hacer el reporte? 'Hacer' es crear algo, y bueno, tienes que buscar específicamente donde se crea el reporte como tal. En este caso sería este. ¿Qué es lo que hace? Busca el método específico de la creación, valida todo lo que está (esto si ya es a nivel de nosotros), pero como tal el propio frontend debería demostrar los valores de los campos específicos para no dar tanto problema. Y una vez que el reporte está validado, simplemente se devuelve eso a través de la misma ruta; es decir, el backend devuelve una respuesta y esa respuesta le llega a ustedes al front nuevamente.

Parte 3: Interacción Frontend-Backend y Preguntas del Equipo

[00:15:53]

Ponente: Simplemente llegarían hasta allí y ya con eso créanme que no les va a hacer más preguntas porque es lo que les concierne a ustedes. No tienen que ir específicamente al proceso de... al método, explicar línea por línea. Dudo mucho que el profesor les llegue a decir eso.

Entonces, bueno, a grandes rasgos eso seria el funcionamiento del back junto con todos estos archivos de controladores; son cada uno de los metodos a los que se pueda acceder y a los que se necesita.

[00:16:32]

Ponente: Y ya con eso... bueno si, ya con eso estaria. Y bueno, una ultima cosa: esto... estas son como tal una manera en la que nosotros definimos como les queremos enviar la informacion a ustedes. Ya aqui esto define una respuesta especifica. Estos Resources no solamente definen los propios datos de la base de datos, sino que tambien definen las relaciones. Y los ayuda a ustedes, al frontend, a que directamente al recibir la respuesta del controlador, la respuesta llegue con toda la informacion necesaria.

[00:17:18]

Ponente: Incluso llegan con informacion extra que puede que no sea necesaria, pero llega. Y si ya en un futuro se decide que si es necesaria esa informacion o se quiere cambiar la vista para que muestre mas informacion, se pueda hacer porque ya directamente les esta enviando toda la informacion. No solamente del controlador especifico, por ejemplo este del activo: no solamente te esta enviando toda la tabla del activo por asi decirlo, sino que tambien te esta enviando toda la informacion del articulo al que pertenece ese activo y la ubicacion al que pertenece ese activo tambien.

[00:17:54]

Ponente: Es decir, es una manera en la que nosotros en un futuro les ayudamos a que pueda crecer el sistema. En general es eso. Asi que si tienen preguntas, que se que si tienen, pueden hacerlo, no hay problema.

[00:18:12]

Compañero: Yo tengo una pregunta. Cuando te referias a los metodos este... POST, DELETE y todo eso, te referias como dijiste a los controladores? No? O sea, esos metodos se encuentran en los controladores de cada uno de los modelos del sistema?

[00:18:26]

Ponente: Eh, si. Basicamente le concierne principalmente a los controladores, pero el frontend tambien los utiliza como tal para enviarlo. Pasa que realmente tendria que estar aqui, ver bien como esta el frontend, pero basicamente ve que aqui le estas diciendo: 'Mira, me vas a construir una respuesta de esta URL'. Okay, te voy a explicar que tiene esa respuesta. La respuesta tiene un metodo. Cual es el metodo? El metodo POST.

[00:18:59]

Ponente: Este metodo POST es el mismo que tiene el backend para el metodo de creacion. Es decir, eso no son nombres que uno les coloca como una variable, sino que son metodos ya definidos por el tipo de API que estamos creando, que es una API RESTful. Entonces el frontend tiene que avisar que quiere enviar una solicitud POST de creacion, y el back ya entiende que cuando le llega un POST es que tiene que ir al metodo de creacion.

[00:19:35]

Ponente: Tambien tiene estos Headers que, bueno, simplemente es para que entienda el backend que es lo que quiere recibir el front y en que formato (en este caso formato JSON). Pero si,

basicamente concierne tanto al back como al front.

[00:19:49]

Compañera: Vale, perfecto, si entiendo. Yo queria hacer una pregunta muy de mi ignorancia, no? Porque claro, al principio cuando yo escuche sobre controladores yo dije: 'No, controladores tienen que ser el CRUD'. Pero claro, cuando estuve viendo el metodo GET y estuve viendo el metodo POST que me dijiste que bueno, tambien es para crear, pero el GET no necesariamente es para crear, es simplemente para como captar esta informacion y tambien te devuelve informacion... entonces nada, ahi si me confundi un poquito.

[00:20:23]

Compañera: Pero lo que quiero dejar claro entonces: en teoria el endpoint seria vamos a decir por ejemplo la URL, que es lo que mas o menos entendi, no? Es decir, hacia donde se conecta la base de datos en el momento en el que el front dice: 'Mira, este es el login, slash lo que sea'. Eso seria lo que capta la informacion. En ese momento entra lo que nosotros conocemos con la vista, luego eso lo estaria manejando los controladores... dijimos que tenemos los recursos para que el backend tenga una respuesta especifica en funcion de lo que la view le esta pidiendo, y los modelos basicamente serian las tablas.

[00:21:11]

Ponente: Si, por lo menos ve este metodo aqui en categorias activos modificar... no se quien habra hecho este. Okay, siempre va a haber un metodo (por lo que he visto en varios que conversaron ustedes) llamado handleSubmit. Ya pueden revisar los del front en su codigo y busquen este metodo porque estoy viendo que es el que utilizaron.

[00:21:43]

Ponente: Yo no he trabajado con el front ni se como funciona cual es la sintaxis de React, pero basicamente estoy viendo que este es el metodo que utilizan ustedes para conectarse al endpoint. Aqui ya estan poniendo categorias/activo. Entonces esta ruta es la que se esta conectando con el back o le esta diciendo al servidor: 'Mira, tengo esta ruta primero y principal, existe?'. Si, si existe. Okay, te estoy mandando un metodo tal, quiero que me devuelvas los resultados.

[00:22:14]

Compañero: Si, ese handleSubmit es basicamente un manejador del envio. Ese es el que controla la accion del submit que esta en el boton alli abajo. El formulario tiene un boton casi siempre que es un submit y alla arriba es la parte que controla eso. Porque toda esta seccion que estas viendo ahorita, eso es vista, eso es puro HTML y CSS, o sea, no tiene tanta logica. Tiene etiquetas que despues en la parte que esta mas arriba es la que referenciamos para poder darle la interaccion con el backend para poder meter variables o sacar datos que ingresa el usuario.

Parte 4: Archivos de Servicios, Axios y Bearer Token

[00:23:22]

Compañera: Yo creo que mas que varias preguntas era como una pregunta muy en general, pues porque queria saber si ya habia como captado cada tema que habias cubierto en la exposicion o en el seminario.

[00:23:39]

Ponente: Si, en general se esta captando la idea. Porque claro, es como yo les digo: lo importante de esto es que no es que tengan que llegar hasta el fondo del back, o sea, no van a llegar hasta la base de datos basicamente. Y una de las cosas que este tipo de arquitectura o forma de ordenar el codigo (que es el que utiliza Laravel por defecto) es precisamente no tocar la base de datos de manera directa con SQL, sino que es simplemente extracciones. Entonces, los modelos como tal son esas abstracciones de la base de datos para que pueda trabajar con ella.

[00:24:27]

Ponente: Y los controladores si reciben esos modelos para poder trabajar tambien con la informacion que tiene la base de datos. Y las vistas se comunican son con los controladores; o sea, no tocan los modelos, sino que confian en que los controladores estan bien hechos y que tienen toda la informacion necesaria para que el modelo responda. Es decir, esta (la vista) no conoce a esta (el modelo), simplemente conoce al controlador.

[00:25:13]

Compañero (Juan): Yo quiero hacer una pregunta con respecto a eso un poquito mas orientado a middleware... a la hora de conectar la vista con el backend, lo conectas a traves de que? Del endpoint? O sea, yo lo que uso es un archivo que se llama service. Por ahi debe estar la carpeta service, que eso lo que tiene son los endpoints.

[00:25:44]

Compañero (Juan): Pero yo lo estaba haciendo al principio de una manera con una funcion fetch. Primero, ahi arriba esta lo que es la URL pues, que simplemente es /api y en este caso dice /catalogo porque ahi estoy trabajando es con los activos. Entonces como que de base ya se que todas mis direcciones en este service van a tener esa URL. Luego lo que cambia es lo que esta despues y el metodo. Y entonces cada una de esas funciones que estan ahi son funciones que van a llamar alguno de los controladores que estan en el backend.

[00:26:24]

Compañero (Juan): Van a usar uno de los protocolos HTTP de cada uno de esos endpoints: los PUT, los POST, los DELETE... alguno de esos metodos voy a usar con alguna URL y obviamente les tengo que mandar una informacion. Aqui, por ejemplo, ese que resalto ahorita tiene el Bearer Token porque es una vista protegida.

[00:26:46]

Compañero (Juan): Y en el backend la tienen protegida. Entonces en el backend esperan que reciba un token para poder devolverte una respuesta, porque si no tiene el token van a mandar un error 401, que es el de que no esta autorizado. Entonces no me va a permitir comunicarme, yo le tengo que mandar el token.

[00:27:05]

Compañero (Juan): Pero esta era la forma en la que la estaba haciendo yo primero. Despues Cesar le incluyo otra cuestion que se llama el Axios, que esta en una funcion api.ts. Eso como que lo centraliza un poco mas. Tipo, ahi ya tiene de base el localhost/api que ya es la base de todo el backend, y centraliza algunos headers. Incluso el Bearer Token tambien esta ahi, pero en otra parte es condicional, depende si hay token o no.

[00:27:52]

Compañero (Juan): Y si ves creo que es el dashboard.service, ahí entonces como Cesar lo estructuro es un poco mas legible porque primero te presenta interfaces de como se presentan los datos. Es un poco mas facil de verlo a simple vista porque ahí ves: 'Bueno, aquí me van a contestar o voy a enviar los datos con esta estructura'. Y aquí en esta otra parte es directamente la comunicacion. Tipo: a donde la voy a mandar y con que metodo?

[00:28:26]

Compañero (Juan): En este caso el dashboard nada mas mandaba puros GET porque lo unico que quiero es mostrar, no quiero mandar informacion. Cuando yo nada mas quiero mostrar algo en el frontend yo uso un metodo GET. Pero cuando yo quiero guardar algo que este ingresando el usuario o alguna informacion, yo uso un metodo POST porque ahí es donde yo quiero que el backend registre una informacion en la base de datos. Cuando yo quiero corregir algo uso un PUT.

[00:28:55]

Compañero (Juan): Que el POST y el PUT la estructura muchas veces va a ser casi la misma en el JSON me refiero, porque los dos mandan informacion al backend. La diferencia es que el POST va a crear un nuevo registro mientras que el PUT va a editar un registro que ya existe. Okay, entonces no voy tan mal.

Parte 5: CRUD, Roles y Seguridad con Tokens

[00:29:17]

Compañera: Entonces los controladores si son como... si es como el CRUD, pero otras cosas. Lo que pasa es que el CRUD es Create, Read, Update y Delete, que esas son las funciones basicas de casi cualquier sistema. Eso esta por la parte de los controladores basicamente. En los endpoints es donde hablamos mas del GET, el PUT, el DELETE... y como los endpoints se comunican al controlador, entonces tu tienes el controlador con el CRUD y al endpoint te comunicas con eso de los GET, los PUT, los DELETE y el se va a comunicar al controlador.

[00:30:01]

Ponente: Claro. Y perdon, aparte... porque los endpoints estan es en la ruta. Si, aparte de eso, aparte de lo del CRUD como tal, por asi decirlo, tambien tienen metodos especificos que hacen otras funciones. Por lo menos... estamos en el controlador del material 10, es decir, de tal articulo quiero descargar el manual de usuario, por ejemplo, que no hemos hecho. Entonces el... aquí en este metodo como tal se encarga de gestionar todo lo que seria esa parte de 'descarga'. Igualmente este endpoint existe en lo que es la ruta.

[00:30:48]

Ponente: Ves que incluso hay unas rutas como que mas especificas que dicen, por ejemplo, /seleccion/finalizar, etcetera. Si uno se va especificamente a este metodo del controlador, es un metodo que no es un CRUD, simplemente es algo que... por asi decirlo, un metodo interno para el funcionamiento del sistema. Estas son reglas de negocio aparte del CRUD normal de toda la vida.

[00:31:21]

Ponente: Son las reglas de negocio que tambien el profesor creo que nos va a pedir, que es para automatizar. Porque si fuese un CRUD basico solo, puede funcionar, si, pero va a haber mucho trabajo manual. En cambio, si se separan ciertas funciones y se delegan y se definen que es lo que

tienen que hacer cuando se realiza cierta accion, se automatiza todo, que es el proposito de cualquier sistema.

[00:31:59]

Compañera: Me imagino que te refieres con... con los procedimientos almacenados, no?

[00:32:05]

Ponente: Eh, si. Solo que eso es mas a nivel de base de datos. Aqui seria a nivel de codigo. Basicamente vamos a verlo asi: como crear un procedimiento almacenado pero dentro del codigo, no en la base de datos? Okay, si, si. En general es eso. Alguna otra pregunta? Alguien mas que quiera preguntar que no sea yo?

[00:32:30]

Compañero: Yo tenia otra pregunta mas sobre lo que dijiste sobre las autorizaciones. Como tal, cuando usted soltaba un token y el codigo que te arrojaba... eso se refiere mas que todo a segun sea el rol que uno tenga en la misma empresa? Por ejemplo, si yo soy solamente un empleado y no soy el administrador y estoy tratando de acceder a informacion de la cual solamente tiene acceso el superior, me deberia arrojar un error porque no me esta dando el token, o como?

[00:33:00]

Ponente: Eso que dijo Juan sobre el Bearer Token que lo menciono... eso es un token. Un token es una secuencia de letras y numeros relarguissimas que identifica a un usuario e identifica la sesion en la que esta. Porque cuando tu te sales, ese token se borra. Entonces ese token, mientras estas en la sesion, identifica tu usuario y por ende identifica que rol tiene.

[00:33:39]

Ponente: Por lo menos aqui se ve claramente en la ruta de la API. Si uno quita esto, 'rutas protegidas', se desaparece casi todo. Lo unico que esta publico-publico que tu puedas acceder en localhost/autenticacion es esto. Ahora, si tu intentas acceder a una ruta protegida, ya aqui te esta diciendo: 'Mira, esta ruta esta siendo protegida por este metodo'. Si tu intentas acceder al dashboard o estadisticas desde la ruta sin registrarte, el te va a dar un error porque no estas autorizado.

[00:34:31]

Ponente: Ahora, si tu inicias sesion como 'reporter' (un usuario cualquiera) e intentas acceder a la misma ruta (pones localhost/api/estadistica), te va a soltar un error de que tampoco puedes porque no estas registrado ni como administrador ni como supervisor. Te da el mismo error 401, creo que es el de autorizacion. Y ya si accedes con el rol especifico, si puedes entrar tranquilamente.

[00:35:07]

Compañero (Juan): Tambien cabe aclarar que la proteccion es doble: esta tanto en el frontend como en el backend. El frontend tambien tiene las vistas protegidas por rol. Porque aja, cuando te logueas, dentro de los datos que devuelve el backend al loguearte, devuelve el rol del usuario. Uno captura eso en lo que seria el contexto o la sesion del usuario (que eso es el local storage) y ahi uno le puede prohibir una vista u otra dependiendo del rol o de si esta logueado o no.

[00:36:00]

Compañero (Juan): En supuesto caso de que encontrases la forma de saber cual es el endpoint del backend del que quieres obtener informacion y extraes tu token de tu sesion, lo mandas por

una aplicacion tipo Insomnia o Postman, o incluso por la terminal con cURL, el backend te lo va a rechazar porque este tiene tambien la proteccion por rol y por autenticacion.

[00:36:40]

Ponente: Okay, perfecto. Vale, se entiende. Gracias. Doble proteccion basicamente. Si, si, esta bien porque igual todos los archivos tienen que estar protegidos.

Parte 6: Seguridad, Librerias de Frontend y Migraciones

[00:37:05]

Ponente: Exacto. Es que si no hay esa proteccion, cualquier persona con un poquito de conocimiento tecnico te tumba la base de datos o te roba informacion sensible. Yo he visto sistemas en casinos donde trabajaba que, por un error de estos, la gente podia ver el saldo de otros jugadores. Asi que aqui estamos aplicando estandares de industria para que eso no pase.

[00:37:37]

Compañera: Claro, totalmente. Oye, una pregunta sobre el frontend, que mencionaron algo de librerias. Que librerias estamos usando especificamente para que el profe vea que estamos actualizados?

[00:38:07]

Compañero (Juan): Bueno, en React estamos usando Lucide React para todo lo que son los iconos y que la interfaz se vea moderna. Tambien usamos React Router para manejar la navegacion sin que la pagina se tenga que recargar cada vez que haces clic en un menu. Y para el estado global, usamos los hooks nativos de React como useContext para que el token de la sesion este disponible en toda la aplicacion.

[00:38:58]

Ponente: Eso es clave. Ahora, volviendo un segundo al backend, hay algo que el profesor seguro les va a preguntar: 'Como crearon las tablas?'. Y ahi es donde entran las Migraciones. Las migraciones son basicamente archivos de PHP que le dicen a la base de datos: 'Crea una tabla llamada activos con estas columnas'.

[00:39:41]

Ponente: Lo bueno de las migraciones es que si nosotros manana cambiamos de base de datos de MySQL a PostgreSQL, no tenemos que reescribir nada. Laravel se encarga de traducir ese codigo PHP al lenguaje de la base de datos que estemos usando. Es como un historial de versiones para tu base de datos.

[00:40:23]

Ponente: Y para consultar esos datos, usamos Eloquent ORM. Eloquent es lo que nos permite hacer algo como `Activo::all()` y eso automaticamente se traduce internamente a un `SELECT * FROM activos`. Es mucho mas limpio y seguro contra ataques de inyeccion SQL, porque Laravel ya limpia los datos por nosotros.

[00:41:15]

Compañera: O sea que si yo quiero buscar un usuario por su ID, no tengo que escribir el codigo SQL largo?

[00:41:25]

Ponente: Exacto. Solo pones `User::find(id)` y listo. El modelo hace todo el trabajo sucio por debajo. Ahora, si ven aquí en la pantalla, les estoy mostrando como se ven las relaciones dentro de las migraciones. Ven que aquí dice `$table->foreignId('articulo_id')`... eso es lo que crea el cable, la conexión entre la tabla de artículos y la de activos.

[00:42:17]

Ponente: Si esa relación no existe en la migración, por más que la pongas en el modelo, la base de datos te va a dar un error de integridad. Por eso el orden es importante: primero creamos las tablas que no dependen de nadie (las maestras) y luego las que tienen llaves foráneas.

[00:43:20]

Compañero (Carlos): Esa es la parte que a veces se me complica, porque si intento borrar un artículo que tiene activos asociados, el sistema me frena, cierto?

[00:43:45]

Ponente: Correcto. Eso es protección de datos. En el código nosotros definimos si queremos que se borre 'en cascada' (que borre todo lo relacionado) o que simplemente tire un error para avisar que ese artículo todavía tiene equipos asignados. Casi siempre preferimos que tire el error para evitar borrar cosas por accidente.

Parte 7: Docker, AuthGuard y Control de Roles en la Interfaz

[00:44:30]

Compañero (Carlos): Esa parte de la protección de integridad es clave para que no nos quede basura en la base de datos. Ahora, una duda técnica: para que el profesor pueda correr esto en su computadora sin instalar mil cosas, como lo tenemos preparado?

[00:45:01]

Ponente: Para eso estamos usando Docker. Básicamente, Docker empaqueta todo: el servidor de PHP, la base de datos MySQL y el servidor de archivos. Así evitamos el famoso: 'En mi computadora sí funcionaba'.

[00:45:40]

Ponente: Hay un archivo llamado `docker-compose.yml` que tiene toda la configuración. Con un solo comando el profesor puede levantar todo el ecosistema. Además, creamos un script de instalación que genera las llaves de seguridad, corre las migraciones y llena la base de datos con datos de prueba (los seeders).

[00:46:15]

Compañera: O sea que los usuarios que ya aparecen creados con sus roles, vienen de esos seeders?

[00:46:25]

Ponente: Exactamente. Tenemos usuarios para cada nivel: un administrador, un supervisor y un técnico. Así, durante la defensa, podemos saltar de uno a otro para mostrar como cambia la interfaz.

[00:47:10]

Compañero (Juan): Y hablando de como cambia la interfaz, ahí es donde entra lo que hicimos en React con el AuthGuard. Si te fijas en el código del frontend, hay un componente que envuelve a las rutas.

[00:47:50]

Compañero (Juan): Este AuthGuard lo que hace es preguntar: 'Tienes un token válido en el local storage?'. Si la respuesta es no, automáticamente te saca de donde estes y te manda al login. No importa si intentaste escribir la URL a mano.

[00:48:30]

Ponente: Y no solo eso. Dentro de una misma página, por ejemplo la lista de activos, usamos el componente RoleRequirement. Esto es genial porque permite que el técnico vea la lista de equipos, pero el botón de 'Eliminar' solo se renderiza si el rol guardado en el sistema es 'Administrador'.

[00:49:15]

Compañera: Eso es lo que yo quería ver. Entonces, aunque compartan la misma vista, la experiencia es diferente.

[00:49:40]

Ponente: Así es. Es una capa de seguridad visual, pero como dijimos antes, reforzada por el backend. Si un técnico 'hackea' el navegador para que aparezca el botón de eliminar y le da clic, el backend recibirá la petición, verá que su token es de técnico y le responderá con un error 403 Forbidden (Prohibido).

[00:50:20]

Compañero (Carlos): Perfecto. Creo que con esta explicación de cómo se conectan los roles desde el token hasta el botón que ve el usuario, cubrimos la parte de seguridad que tanto le importa al jurado.

Parte 8: Cierre - Variables de Entorno, Enums y Consejos Finales

[00:51:15]

Ponente: Ya para ir cerrando esta sesión, quiero que se fijen en la carpeta lib y en services dentro del frontend. Cesar hizo un gran trabajo ahí centralizando todo con Axios. Si el día de mañana el servidor cambia de IP o de dominio, solo tenemos que cambiar una línea en el archivo .env y todo el sistema seguirá funcionando perfectamente.

[00:52:01]

Compañera: Eso es lo que llaman variables de entorno, cierto?

[00:52:12]

Ponente: Exacto. Nunca, pero nunca, deben escribir la dirección del servidor directamente en los componentes. Siempre se usa la variable de entorno por seguridad y flexibilidad.

[00:52:45]

Compañero (Juan): También es bueno mencionar que en el frontend estamos manejando los errores de red de forma amigable. Si el backend se cae o hay un problema de conexión, el sistema lanza una alerta (un Toast) que le dice al usuario: 'Error de conexión, inténtelo más tarde', en lugar de simplemente quedarse la pantalla en blanco o que la aplicación explote.

[00:53:20]

Ponente: Buen punto. Ahora, sobre la entrega final: el repositorio esta organizado de manera que el profesor pueda ver el historial de commits. Eso demuestra que no lo hicimos de un dia para otro, sino que hubo un proceso de desarrollo real.

[00:54:05]

Compañera: Y que pasa si nos preguntan sobre los Enums? Vi que hay una carpeta con ese nombre.

[00:54:15]

Ponente: Los Enums (Enumeraciones) los usamos para estandarizar los estados. Por ejemplo, en lugar de escribir 'Mantenimiento' con la primera letra en mayuscula en un lado y en minuscula en otro, el Enum obliga a que siempre se use la misma palabra exacta. Asi evitamos errores de logica cuando filtramos activos en el dashboard.

[00:55:30]

Compañero (Carlos): Muchachos, creo que estamos listos. El sistema es solido, tiene seguridad por roles, arquitectura MVC y esta dockerizado. Si cada uno domina su parte como lo hablamos hoy, no deberiamos tener problemas.

[00:56:45]

Ponente: Ultimo consejo: si el profesor se pone muy tecnico con la base de datos, siempre lleven la conversacion hacia el Eloquent ORM. Expliquen que Laravel gestiona esa capa para que nosotros nos enfoquemos en la logica de negocio del inventario y el mantenimiento. Eso les da un perfil de desarrolladores mas profesionales.

[01:02:10]

Compañera: Vale, me siento mucho mas segura ahora. Gracias por tomarte el tiempo de explicarnos el back a los que estamos mas pegados con el diseno.

[01:05:40]

Ponente: No hay de que. Somos un equipo. Bueno, voy a detener la grabacion. Revisen este video si tienen dudas mientras estudian el codigo. Mucho exito en la defensa!

[01:09:14]

—: (Fin del audio y de la grabacion de la reunion)